Taller de Cálculo y Álgebra Relacional

Estudiantes destacados de Ingeniería de Sistemas, período 2026-1

El enunciado pide la consulta en **Cálculo Relacional de Tuplas (TRC)**, pero la notación que usamos en clase es la del **álgebra relacional** (Π, σ, ×). Por eso el documento resuelve el ejercicio de las dos maneras: primero en TRC, que es lo que se solicita, y después en álgebra con la notación del curso. Así se puede comparar cómo se dice lo mismo en un lenguaje declarativo y en uno procedimental.

Convención de la clase: donde otros textos escriben `∧` dentro de una selección, aquí se puede anidar una selección dentro de otra. `σA(σB(R))` equivale a `σA∧B(R)`. En el desarrollo uso la forma anidada para que cada paso se lea por separado.

---

1. Operadores

1.1 Álgebra relacional

Operador	Qué hace	Restricciones

σ Selección	Deja las tuplas que cumplen una condición	Los atributos de la condición deben existir en la relación. Los valores comparados deben ser de dominios compatibles.
Π Proyección	Deja solo las columnas indicadas	Los atributos deben existir. El resultado elimina duplicados, porque una relación es un conjunto.
× Producto cartesiano	Combina cada tupla de R con cada tupla de S	Si dos relaciones comparten nombre de atributo hay que prefijarlo (`E.id_carrera`, `C.id_carrera`). Produce |R|·|S| tuplas, así que casi siempre va seguido de σ.
⋈ Reunión natural	Producto más igualdad de los atributos con el mismo nombre	Debe haber al menos un atributo común con el mismo dominio. Si no lo hay, se degrada a producto cartesiano.
∪ Unión	Tuplas que están en R, en S o en ambas	R y S deben ser **compatibles para la unión**: mismo número de atributos y dominios correspondientes.
– Diferencia	Tuplas de R que no están en S	Misma compatibilidad. No es conmutativa: R – S ≠ S – R.
∩ Intersección	Tuplas que están en R y en S	Misma compatibilidad. Se puede escribir como R – (R – S).
ρ Renombre	Cambia el nombre de una relación o de sus atributos	El nombre nuevo no puede chocar con otro que se use en la misma expresión. Es indispensable para el producto de una relación consigo misma.

← Asignación	Guarda un resultado intermedio con nombre	Solo da nombre. No agrega poder de expresión, solo legibilidad.

Una limitación que pesa en este ejercicio: el álgebra relacional pura **no tiene COUNT**. Contar "al menos tres" obliga a combinar la relación consigo misma y exigir que las tuplas sean distintas.

1.2 Cálculo relacional de tuplas

Símbolo	Se lee	Restricciones
`{ t.a, t.b | F(t) }`	"las tuplas con esos atributos tales que F"	Las variables a la izquierda de | son las únicas libres en F.
`R(t)`	"t pertenece a la relación R"	Toda variable debe quedar acotada a una relación. Sin eso la consulta no es *segura* y podría devolver infinitos resultados.
∃t (F)	existe una tupla t tal que F	La variable solo vale dentro de los paréntesis del cuantificador.
∀t (F)	para toda tupla t, F	Se usa casi siempre con implicación: `∀t (R(t) → F)`.
¬	negación	Se aplica a una fórmula completa.
∧, ∨	y, o	Sin restricción especial.
→	si... entonces	`P → Q` equivale a `¬P ∨ Q`.
=, ≠, <, ≤, >, ≥	comparaciones	Dominios compatibles entre los dos lados.

Dos equivalencias que se usan abajo y conviene tener a mano:

- `∀x (P → Q)` es lo mismo que `¬∃x (P ∧ ¬Q)`.
- No existe una operación de conteo. "Al menos tres" se expresa

con tres variables y desigualdades entre ellas.

---

2. Relaciones del ejercicio

Cinco relaciones. **Negrita** marca la clave primaria, *cursiva* las claves foráneas.

Relación	Atributos
CARRERA	**id_carrera**, nombre, facultad
ESTUDIANTE	**id_estudiante**, nombre, apellido, edad, correo, *id_carrera*
MATRICULA	**id_estudiante**, **id_curso**, semestre, nota_final, estado
CURSO	**id_curso**, nombre_curso, creditos, *id_carrera*, *id_profesor*
PROFESOR	**id_profesor**, nombre, apellido, especialidad

Cómo se conectan:

- Una carrera tiene muchos estudiantes y muchos cursos.
- MATRICULA es la tabla puente entre ESTUDIANTE y CURSO (relación N:M).
- Cada curso lo dicta un profesor, y un profesor puede dictar varios cursos.

---

3. Enunciado

La Universidad quiere identificar a los estudiantes de **Ingeniería de Sistemas** con desempeño destacado en **2026-1**. Deben cumplir, todas a la vez:

1. Pertenecer a Ingeniería de Sistemas.
2. Estar matriculados en **al menos tres cursos** en 2026-1.
3. Tener **una nota mayor a 4.0** en al menos uno de esos cursos.
4. **No haber perdido ninguno**. Un curso se pierde con nota final menor a 3.0.
5. Haber aprobado **todos los cursos de 3 créditos de su carrera** que matricularon en 2026-1.
6. Si un curso de 3 créditos lo dicta un profesor con especialidad **"Bases de Datos"**, la nota debe ser **mayor o igual a 3.5**.
7. Tener **al menos un curso de 4 o más créditos aprobado** en 2026-1.

Se pide el nombre, apellido y correo de esos estudiantes.

---

4. Solución en Cálculo Relacional de Tuplas

La idea general es partir de cada estudiante `e` y filtrarlo con siete condiciones unidas por `∧`. Cada condición se traduce a un cuantificador. Las existenciales piden que algo exista, las negaciones piden que algo no exista y la universal revisa que una regla se cumpla en todos los casos.

El camino entre tablas es: ESTUDIANTE → CARRERA, ESTUDIANTE → MATRICULA → CURSO → PROFESOR.

Paso 1. Pertenece a Ingeniería de Sistemas

Se busca la carrera del estudiante y se verifica el nombre.

```
∃ca ( CARRERA(ca) ∧ ca.id_carrera = e.id_carrera
      ∧ ca.nombre = "Ingeniería de Sistemas" )
```

Paso 2. Al menos tres cursos en 2026-1

No hay conteo, así que se piden **tres matrículas distintas** del mismo estudiante en el mismo semestre. La distinción se garantiza comparando los `id_curso` de las tres entre sí.

```
∃m1 ( MATRICULA(m1) ∧ m1.id_estudiante = e.id_estudiante
  ∧ ∃m2 ( MATRICULA(m2) ∧ m2.id_estudiante = e.id_estudia
          ∧ m2.id_curso ≠ m1.id_curso
    ∧ ∃m3 ( MATRICULA(m3) ∧ m3.id_estudiante = e.id_estud
            ∧ m3.id_curso ≠ m1.id_curso ∧ m3.id_curso ≠ m
```

Las tres desigualdades (m1≠m2, m1≠m3, m2≠m3) son necesarias. Si falta una, dos de las variables podrían apuntar a la misma matrícula y el conteo sería falso.

Paso 3. Alguna nota mayor a 4.0

```
∃m4 ( MATRICULA(m4) ∧ m4.id_estudiante = e.id_estudiante
      ∧ m4.semestre = "2026-1" ∧ m4.nota_final > 4.0 )
```

Paso 4. Ningún curso perdido

Aquí cambia el cuantificador. "Ninguno" se dice con **no existe** una matrícula con nota menor a 3.0.

```
¬∃m5 ( MATRICULA(m5) ∧ m5.id_estudiante = e.id_estudiante
       ∧ m5.semestre = "2026-1" ∧ m5.nota_final < 3.0 )
```

Paso 5. Aprobó todos sus cursos de 3 créditos

Es una regla del tipo "si matriculó un curso de 3 créditos de su carrera, entonces lo aprobó". Eso es un `∀` con implicación.

```
∀c ( ( CURSO(c) ∧ c.id_carrera = e.id_carrera ∧ c.credito
       ∧ ∃mx ( MATRICULA(mx) ∧ mx.id_estudiante = e.id_es
```

```
                ∧ mx.id_curso = c.id_curso ∧ mx.semestre =
      →
      ∃my ( MATRICULA(my) ∧ my.id_estudiante = e.id_estudi
            ∧ my.id_curso = c.id_curso ∧ my.semestre = "20
            ∧ my.nota_final ≥ 3.0 ) )
```

Antes de la flecha va la condición de aplicabilidad (es un curso de 3 créditos de su carrera y lo matriculó). Después va lo que debe cumplirse. Los cursos que no cumplen el antecedente no molestan: la implicación es verdadera por vacío.

Paso 6. Cursos de 3 créditos con profesor de Bases de Datos

Misma estructura, con un filtro más en el antecedente (el profesor) y una nota más exigente en el consecuente.

```
∀c2 ( ( CURSO(c2) ∧ c2.id_carrera = e.id_carrera ∧ c2.cre
        ∧ ∃p ( PROFESOR(p) ∧ p.id_profesor = c2.id_profes
               ∧ p.especialidad = "Bases de Datos" )
        ∧ ∃m6 ( MATRICULA(m6) ∧ m6.id_estudiante = e.id_e
                ∧ m6.id_curso = c2.id_curso ∧ m6.semestre
      →
      ∃m7 ( MATRICULA(m7) ∧ m7.id_estudiante = e.id_estud
            ∧ m7.id_curso = c2.id_curso ∧ m7.semestre = "
            ∧ m7.nota_final ≥ 3.5 ) )
```

Paso 7. Al menos un curso de 4 o más créditos aprobado

Aquí hay que cruzar MATRICULA con CURSO para saber los créditos.

```
∃m8 ( MATRICULA(m8) ∧ m8.id_estudiante = e.id_estudiante
      ∧ m8.semestre = "2026-1" ∧ m8.nota_final ≥ 3.0
      ∧ ∃c3 ( CURSO(c3) ∧ c3.id_curso = m8.id_curso ∧ c3.
```

Consulta completa

```
{ e.nombre, e.apellido, e.correo |
    ESTUDIANTE(e)
  ∧ ∃ca ( CARRERA(ca) ∧ ca.id_carrera = e.id_carrera
          ∧ ca.nombre = "Ingeniería de Sistemas" )
  ∧ ∃m1 ( MATRICULA(m1) ∧ m1.id_estudiante = e.id_estudia
    ∧ ∃m2 ( MATRICULA(m2) ∧ m2.id_estudiante = e.id_estud
            ∧ m2.id_curso ≠ m1.id_curso
      ∧ ∃m3 ( MATRICULA(m3) ∧ m3.id_estudiante = e.id_est
              ∧ m3.id_curso ≠ m1.id_curso ∧ m3.id_curso ≠
  ∧ ∃m4 ( MATRICULA(m4) ∧ m4.id_estudiante = e.id_estudia
          ∧ m4.semestre = "2026-1" ∧ m4.nota_final > 4.0
  ∧ ¬∃m5 ( MATRICULA(m5) ∧ m5.id_estudiante = e.id_estudi
           ∧ m5.semestre = "2026-1" ∧ m5.nota_final < 3.0
  ∧ ∀c ( ( CURSO(c) ∧ c.id_carrera = e.id_carrera ∧ c.cre
           ∧ ∃mx ( MATRICULA(mx) ∧ mx.id_estudiante = e.i
                   ∧ mx.id_curso = c.id_curso ∧ mx.semest
         → ∃my ( MATRICULA(my) ∧ my.id_estudiante = e.id_
                 ∧ my.id_curso = c.id_curso ∧ my.semestre
                 ∧ my.nota_final ≥ 3.0 ) )
  ∧ ∀c2 ( ( CURSO(c2) ∧ c2.id_carrera = e.id_carrera ∧ c2
            ∧ ∃p ( PROFESOR(p) ∧ p.id_profesor = c2.id_pr
                   ∧ p.especialidad = "Bases de Datos" )
            ∧ ∃m6 ( MATRICULA(m6) ∧ m6.id_estudiante = e.
                    ∧ m6.id_curso = c2.id_curso ∧ m6.seme
          → ∃m7 ( MATRICULA(m7) ∧ m7.id_estudiante = e.id
                  ∧ m7.id_curso = c2.id_curso ∧ m7.semest
                  ∧ m7.nota_final ≥ 3.5 ) )
  ∧ ∃m8 ( MATRICULA(m8) ∧ m8.id_estudiante = e.id_estudia
          ∧ m8.semestre = "2026-1" ∧ m8.nota_final ≥ 3.0
          ∧ ∃c3 ( CURSO(c3) ∧ c3.id_curso = m8.id_curso ∧
}
```

---

5. Solución en álgebra relacional (notación de clase)

En álgebra no hay cuantificadores, así que la estrategia cambia. Se calcula, para cada condición, **el conjunto de `id_estudiante` que la cumple** y al final se intersectan todos. Las condiciones negativas ("ninguno", "todos") se resuelven con **diferencia**: se toma a todos los estudiantes y se restan los que fallan.

Todos los conjuntos intermedios tienen un solo atributo, `id_estudiante`, y por eso son compatibles para ∩ y −.

Paso 0. Insumos base

Se filtra la matrícula del período una sola vez y se reutiliza.

```
M26 ← σsemestre='2026-1'(MATRICULA)
TODOS ← Πid_estudiante(ESTUDIANTE)
```

Paso 1. Estudiantes de Ingeniería de Sistemas

```
C1 ← ΠE.id_estudiante(σE.id_carrera=CA.id_carrera(σCA.nom
```

Primero se filtra el nombre de la carrera, después se amarra la clave foránea. Si se invirtiera el orden el resultado sería igual, pero se manejarían más tuplas intermedias.

Paso 2. Al menos tres cursos

Se toman tres copias de M26 con ρ. Se exige que sean del mismo estudiante y de cursos distintos entre sí.

```
M1 ← ρM1(M26)     M2 ← ρM2(M26)     M3 ← ρM3(M26)

C2 ← ΠM1.id_estudiante(
        σM2.id_curso≠M3.id_curso(
        σM1.id_curso≠M3.id_curso(
        σM1.id_curso≠M2.id_curso(
```

```
        σM2.id_estudiante=M3.id_estudiante(
        σM1.id_estudiante=M2.id_estudiante(
           M1 × M2 × M3))))))
```

Quien tenga cuatro o más cursos también aparece, porque basta con encontrar una terna de cursos distintos.

Paso 3. Alguna nota superior a 4.0

```
C3 ← Πid_estudiante(σnota_final>4.0(M26))
```

Paso 4. Ningún curso perdido

Se calculan los que **sí** perdieron alguno y se restan del total.

```
PERDIERON ← Πid_estudiante(σnota_final<3.0(M26))
C4 ← TODOS − PERDIERON
```

Paso 5. Aprobó todos sus cursos de 3 créditos

Se buscan los que reprobaron un curso de 3 créditos de su propia carrera y se les resta del total.

```
REPRUEBAN3 ← ΠE.id_estudiante(
                σM.nota_final<3.0(
                σC.creditos=3(
                σC.id_carrera=E.id_carrera(
                σM.id_curso=C.id_curso(
                σM.id_estudiante=E.id_estudiante(
                   ESTUDIANTE × M26 × CURSO))))))
C5 ← TODOS − REPRUEBAN3
```

Se ve el valor de este paso: `REPRUEBAN3` es un subconjunto de `PERDIERON`. Ver la nota 1 del apartado 6.

Paso 6. Cursos de 3 créditos con profesor de Bases de Datos

Misma idea, con PROFESOR en el producto y el umbral de 3.5.

```
FALLAN_BD ← ΠE.id_estudiante(
                σM.nota_final<3.5(
                σP.especialidad='Bases de Datos'(
                σP.id_profesor=C.id_profesor(
                σC.creditos=3(
                σC.id_carrera=E.id_carrera(
                σM.id_curso=C.id_curso(
                σM.id_estudiante=E.id_estudiante(
                   ESTUDIANTE × M26 × CURSO × PROFESOR))))
C6 ← TODOS − FALLAN_BD
```

Paso 7. Un curso de 4 o más créditos aprobado

```
C7 ← ΠM.id_estudiante(
        σC.creditos≥4(
        σM.nota_final≥3.0(
        σM.id_curso=C.id_curso(
           M26 × CURSO))))
```

Paso 8. Intersección y proyección final

```
IDS ← C1 ∩ C2 ∩ C3 ∩ C4 ∩ C5 ∩ C6 ∩ C7

RESULTADO ← ΠE.nombre, E.apellido, E.correo(σE.id_estudia
```

El último paso recupera los datos personales. Los pasos anteriores trabajan solo con identificadores porque así las operaciones de conjuntos son válidas.

---

6. Observaciones para la revisión

1. **La condición 5 es redundante.** Si nadie del resultado tiene una nota menor a 3.0 (condición 4), tampoco puede haber reprobado un curso de 3 créditos. La condición 6 sí aporta, porque sube el umbral a 3.5 en un caso particular. El enunciado las pide por separado, así que se dejaron las dos, pero vale la pena decirlo en clase.
2. **Llave de MATRICULA.** Con la llave (id_estudiante, id_curso) un estudiante no podría repetir el mismo curso en otro semestre. Si el modelo real permite repetirlo, `semestre` debería entrar en la llave. Las consultas de arriba funcionan igual en ambos casos.
3. **Notas nulas.** Un curso matriculado sin nota final no cumple `< 3.0` ni `≥ 3.0`. El ejercicio no lo trata, así que se asume que todas las matrículas del período ya tienen nota.
4. **Estado de la matrícula.** El atributo `estado` no se usa. La aprobación se define solo por la nota, como indica el enunciado.
5. **Corrección frente al borrador.** La solución original quedaba sin la llave final `}` que cierra el conjunto, y faltaba la relación de las preguntas finales (nombre, apellido y correo). Ambas cosas quedan completas arriba.